

Informe técnico del prototipo “Agente Belleza CO”

Agente de inteligencia artificial para búsqueda, comparación y recomendación
de compra de productos de maquillaje entre proveedores

Actividad: “Un agente de IA para un problema real” · Consultoría · 2026-II

Para: Grupo(s) de la actividad de consultoría — Pregrado en Estadística
De: Javier Mauricio Sierra
Fecha: 25 de agosto de 2026
Repositorio: https://github.com/ustadistica/Agente-de-prueba_2026II
Asunto: Justificación, objetivos, construcción y funcionamiento del agente

Este informe documenta el prototipo construido como material de referencia de la actividad. No es un sustituto del trabajo del grupo: es un caso completo—con sus decisiones explícitas, sus pruebas y sus límites declarados—que pueden estudiar, replicar y mejorar. La estructura sigue los apartados que la rúbrica evalúa, de modo que sirve también como modelo para redactar la propia memoria técnica.

1. Resumen ejecutivo

Agente Belleza CO es un agente conversacional con ciclo de decisión propio que ayuda a emprendimientos de belleza a decidir *dónde comprar*: compara un mismo producto entre tiendas online colombianas (modalidad *detal*) y catálogos de distribuidores mayoristas (modalidad por volumen con cantidades mínimas), calcula el costo total incluyendo envío y recomienda la opción más conveniente según los criterios del usuario (presupuesto, unidades objetivo, preferencia de modalidad, ciudad de entrega). El prototipo está operativo, versionado en un repositorio público, verificado con cuatro pruebas automatizadas del ciclo de decisión y desplegable sin instalación mediante Streamlit Community Cloud. Sus límites están declarados en la sección 6: trabaja sobre capturas de datos actualizables y no transacciona.

2. Justificación

El problema y su contraparte

Los emprendimientos dedicados a la comercialización de maquillaje compran inventario bajo dos regímenes de precio que coexisten en el mercado: el *detal* (una unidad, precio de lista en tiendas online) y el *mayorista* (descuento por volumen condicionado a cantidades mínimas que cada distribuidor fija a su arbitrio). La decisión de compra exige consolidar información dispersa—nombres comerciales inconsistentes entre vendedores, precios distintos por tono o presentación, costos de envío variables según ciudad de despacho, stock limitado—y hacerla comparable. Esa comparación se hace hoy manualmente.

La contraparte real del proyecto será [*nombre, rol y negocio*], quien resolvió esta duda por última vez [*frecuencia*] y estima dedicarle [*tiempo por cotización*]. La línea base se medirá cronometrando ese proceso antes de cualquier uso del agente; la misma métrica se repetirá después para cuantificar el aporte (sección 6).

Por qué esto requiere un agente y no un script

El umbral que separa un agente de una función está en el tipo de decisión involucrada. Aquí hay dos que un condicional no puede tomar:

1. **Agrupación de equivalentes.** El mismo labial aparece como “Labial líquido Maybelline SuperStay Matte Ink 5 ml tono 80 Ruler” (Sephora), “Maybelline Superstay Matte Ink Labial Líquido 5ml” (MercadoLibre) o “Labial SuperStay Matte Ink Maybelline varios tonos” (distribuidor). Decidir que son el mismo producto—y que una paleta genérica “Naked” sin marca confirmada *no* equivale a una paleta Huda Beauty—es un juicio semántico dependiente del contexto, no una coincidencia de cadenas.
2. **Recomendación bajo criterios difusos.** Elegir entre pagar menos por unidad comprometiendo capital en doce unidades, o pagar más conservando flexibilidad, depende de presupuesto, rotación del negocio y tolerancia al riesgo del usuario: criterios que se expresan en lenguaje natural y se ponderan distinto en cada consulta.

Un script determinístico resuelve búsquedas fijas; el agente interpreta la solicitud, decide qué consultar, agrupa, calcula, justifica con cifras y deja traza auditable de cada paso. Cuando la consulta es ambigua o los datos insuficientes, lo declara en lugar de suponer.

Anclaje disciplinar

El problema es, en su núcleo, estadístico: *decisión bajo incertidumbre con restricciones*. Intervienen estimación de costo unitario efectivo (costo total dividido entre unidades realmente obtenidas), comparación de alternativas con atributos múltiples (precio, calidad percibida vía calificaciones, tiempo de despacho, riesgo de stock), y sensibilidad de la recomendación a los parámetros del usuario (el ejemplo de la sección 5.2 cambia de respuesta si el presupuesto baja de \$500.000 a \$400.000). La validación del sistema usa el mismo aparato conceptual del curso: casos de prueba con resultados esperados, fallos documentados y métricas comparables contra línea base.

3. Objetivos y alcance

Objetivo general

Construir un agente de IA que automatice la búsqueda, normalización y comparación de productos de maquillaje entre proveedores colombianos, generando un comparativo de costos totales y una recomendación fundamentada según criterios definidos por el usuario.

Objetivos específicos

Cada objetivo quedó materializado en un componente verificable del repositorio:

1. **Consultar fuentes de datos existentes y accesibles:** listado capturado de tiendas online (26

referencias, archivo JSON) y catálogo de distribuidores mayoristas (18 referencias, CSV), ambos editables por la contraparte sin tocar código.

2. **Identificar productos iguales o similares entre proveedores:** agrupación semántica realizada por el LLM dentro de un contrato de salida estructurado.
3. **Comparar precios detal vs. mayorista y respetar cantidades mínimas:** herramienta determinística de cálculo que aplica el precio mayorista solo cuando se alcanza el mínimo exigido.
4. **Incorporar atributos relevantes:** marca, presentación, tono, calificación y número de reseñas, stock, ciudad de despacho y costo de envío entran al comparativo.
5. **Generar tablas comparativas y recomendación fundamentada:** salida JSON validada que la interfaz convierte en tabla exportable a CSV y tarjeta de recomendación citando cifras.
6. **Permitir filtros del usuario:** presupuesto máximo, unidades objetivo, modalidad preferida, marcas de interés y ciudad de entrega, configurables desde la barra lateral.
7. **Garantizar operación segura y reproducible:** credenciales solo por variables de entorno o secrets del panel de despliegue; README con instalación probada desde cero.

Alcance acordado (qué sí y qué no)

Sí incluye: comparación detal/mayorista de productos del catálogo, recomendación con justificación cuantitativa, trazabilidad de decisiones, exportación de resultados.

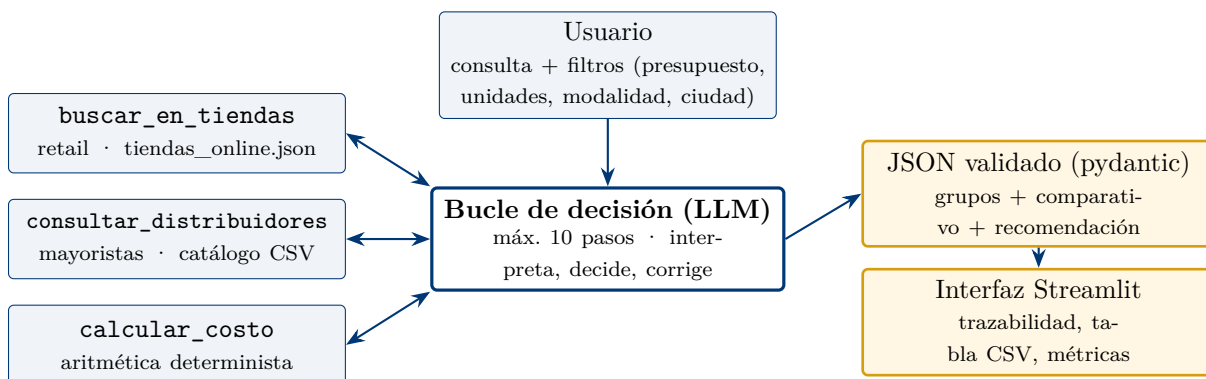
No incluye: compras automáticas ni pagos, navegación web en tiempo real (los datos provienen de capturas actualizables; decisión justificada en la Tabla 1), integración con sistemas de venta de la contraparte, procesamiento de datos personales de clientes finales.

Con la contraparte definitiva deberá formalizarse este alcance en la constancia del encargo, ajustando los términos a lo que ella reconozca como útil.

4. Construcción del agente

Arquitectura

El sistema es un *ciclo único de decisión con herramientas* (no multiagente): un LLM recibe la solicitud con sus filtros, decide qué herramientas invocar, observa los resultados, y repite hasta poder emitir una respuesta estructurada validada contra un esquema estricto.



Las flechas dobles indican el ciclo real de invocación: el agente llama a una herramienta, observa su resultado y decide el siguiente paso (hasta 10 iteraciones).

Decisiones de diseño y alternativas descartadas

Es la parte más importante del documento: toda decisión técnica relevante, contra qué alternativa se descartó y por qué (Tabla 1).

Decisión	Alternativa descartada	Justificación
Datos existentes y editables (JSON + CSV capturados)	Scraping en vivo de tiendas	Evita fragilidad ante bloqueos anti-bot, permisos de terceros y cambios de maquetado; cumple el alcance de “datos ya existentes”; la contraparte puede actualizarlos sin programar.
Ciclo único con tres herramientas	Arquitectura multiagente	El problema cabe en un ciclo; agentes múltiples añaden latencia, costo y puntos de falla sin beneficio verificable en este alcance.
Matching semántico por LLM	Regex o distancia de cadenas	Los nombres comerciales varían demasiado; las técnicas léxicas producen falsos positivos graves (agrupar una réplica genérica con la marca original) y falsos negativos sistemáticos.
Aritmética delegada en herramienta	Cálculos hechos por el LLM	Los LLM fallan multiplicaciones encadenadas; <code>calcular_costo</code> es determinista, auditable y reutilizable. El modelo nunca “inventa” totales.
Contrato JSON validado con pydantic	Texto libre en markdown	Permite renderizar tabla y exportar CSV automáticamente; detecta salidas inválidas y las devuelve al modelo con corrección guiada; incluye validación cruzada (la opción recomendada debe existir en el comparativo).
Gemini vía API compatible OpenAI (configurable)	gpt-4o-mini · Ollama local	Costo cero en capa gratuita y buena relación calidad/latencia para la demo; el código no cambia entre proveedores (solo <code>.env</code>): OpenAI ofrece mejor razonamiento y Ollama privacidad total, a costa de pago local o requisitos de RAM.

Tabla 1. Decisiones de diseño del prototipo.

Datos y fuentes

Dos archivos planos constituyen toda la fuente de verdad del agente, con fecha de captura declarada en sus metadatos: `data/tiendas_online.json` (Sephora Colombia, Falabella.com, MercadoLibre y tienda oficial, con precio detal, calificaciones, stock, ciudad de despacho y envío) y `data/catalogo_distribuidores.csv` (precio detallista, precio mayorista, cantidad mínima, stock máximo y despacho por distribuidor). Los conjuntos incluyen deliberadamente casos difíciles: publicaciones agotadas, cajas cerradas sin precio unitario detal, stock insuficiente frente a la cantidad mínima, y una publicación genérica sin marca confirmada marcada como posible réplica.

El prompt versionado

El comportamiento del agente vive en `prompts/sistema_v1.txt`, cargado por el código y documentado en `prompts/prompts_v1.md` con rol, contrato de salida, restricciones y la justificación de cada decisión de redacción. Al modificarse, se crea `sistema_v2.txt` sin sobrescribir el anterior: la iteración queda evidenciada, como exige la actividad. Entre las restricciones clave: prohibido inventar productos o precios; obligatoriedad de reformular la búsqueda antes de declarar vacío; derivar a *datos insuficientes* lo que no pueda confirmarse; y justificar siempre con cifras concretas.

Manejo de errores por tipo

Tabla 2 resume la política de fallo controlado: ningún error termina en traceback; todos degradan informando al usuario o devolviendo control al modelo.

Tipo de error	Manejo	Resultado observable
Timeout del proveedor LLM	Reintento automático (hasta 3)	Continúa sin intervención.
Límite de peticiones (rate limit)	Retroceso exponencial 2–4–8 s	Mensaje claro si persiste.
Credenciales ausentes/rechazadas	Falla inmediata con instrucciones	Guía de configuración, nunca clave en código.
Herramienta inexistente o argumentos no parseables	El error se devuelve al modelo como observación	Autocorrección dentro del ciclo.
Salida fuera del contrato JSON	Corrección guiada (máx. 2 reintentos)	JSON válido o error explicado.
Recomendación de opción inexistente	Validación cruzada del esquema	Se exige citar una fila real del comparativo.
Fila malformada en el CSV de distribuidores	Validación por campo con identificación de fila y columna	Mensaje accionable de corrección del dato.
Consulta que agota los 10 pasos	Parada controlada	Diagnóstico sugerido (simplificar la consulta).

Tabla 2. Errores gestionados por tipo en `src/llm.py`, `src/agent.py` y `src/tools.py`.

Seguridad, credencialidad y reproducibilidad

Las claves nunca están en el repositorio: localmente en `.env` (ignorado por git) y en la nube en el panel Secrets de Streamlit Cloud; `.env.example` documenta las variables sin valores. Las dependencias están fijadas por versión en `requirements.txt` y la instalación desde cero fue probada en un entorno virtual limpio: clonar, instalar, configurar y ejecutar son cuatro comandos documentados en el README. Cada ejecución queda registrada en la interfaz con el detalle de qué herramientas se llamaron y con qué argumentos: la decisión es auditable después de tomada.

5. Funcionamiento

Flujo de uso

El usuario describe qué necesita (“necesito 12 labiales SuperStay para reventa”), ajusta filtros en la barra lateral y ejecuta. El agente muestra primero su trazabilidad (qué buscó en cada fuente,

qué calculó), luego las métricas de la opción ganadora, la recomendación justificada con cifras, la tabla comparativa descargable, los grupos de equivalentes identificados y las alertas de datos insuficientes. En despliegue, la URL pública de Streamlit Community Cloud permite usar todo esto desde el navegador, sin instalación.

Sesión documentada (caso CP-01)

Con la consulta “Necesito 12 labiales Maybelline SuperStay Matte Ink para reventa” y criterio de menor costo total, el agente agrupa seis referencias de tres fuentes distintas y produce el siguiente comparativo (valores reales del repositorio de datos):

Opción	Modalidad	Costo total	Unitario ef.	Mínimo	Despacho
MercadoLibre (ret-002)	Detal × 12	\$656.500	\$54.708	1	Medellín
Belleza Express (dis-001)	Mayorista × 12	\$468.000	\$39.000	12 u.	Bogotá
Belleza Express (dis-002)	Caja cerrada × 12	—	\$38.000	12 u.	Bogotá

Tabla 3. Comparativo resultante del caso CP-01. La caja dis-002 es la más barata por unidad, pero su stock (10) no cubre las 12 unidades requeridas: el agente la reporta como limitación en lugar de recomendarla.

La recomendación emitida fue la compra mayorista a Belleza Express (dis-001): ahorro de \$188.500 frente al detal en MercadoLibre (28,7 % por unidad), advirtiendo el mínimo de 12 unidades y que el despacho sale desde Bogotá. Si el presupuesto máximo se ajusta a \$400.000, la misma consulta cambia de respuesta: ninguna opción mayorista cabe y el agente debe relegar el detal o reportar que nada cumple (caso CP-02). Esta sensibilidad a los parámetros es exactamente el tipo de razonamiento que se espera examinar en la sustentación.

Casos de prueba y fallos esperados

El protocolo completo está en `docs/casos_prueba.md`; aquí interesa destacar que incluye fallos *por diseño*. CP-03 pide “paletas Naked baratas”: la única coincidencia es una publicación genérica sin marca y con alerta de posible réplica, y el resultado correcto es que el agente *no* la agrupe con marcas reales y la derive a datos insuficientes con advertencia de riesgo. CP-04 solicita un producto inexistente en las fuentes, donde lo correcto es reformular la búsqueda, agotar intentos y declarar que no hay datos—jamás inventarlos. Adicionalmente, cuatro pruebas automatizadas del ciclo de decisión (`tests/test_bucle_stub.py`) se ejecutan sin red ni API key y verifican: invocación correcta de herramientas, manejo de herramienta inválida, recuperación ante salida no parseable y rechazo controlado de un CSV malformado; el módulo de bitácora tiene pruebas propias (`tests/test_bitacora.py`: registro, lectura tolerante a corrupción, métricas y clasificación de errores). Estado honesto de la verificación: la mecánica está probada con LLM simulado; la ejecución de los casos con un modelo real, con registro de salidas, es trabajo pendiente del grupo (sección 6).

Bitácora de ejecución

Toda ejecución del agente—exitosa o fallida—queda registrada automáticamente como evidencia auditable en `bitacora/ejecuciones.jsonl` (una línea JSON por corrida), gracias a `src/bitacora.py`. Registrar también los fracasos es deliberado: sin errores en la muestra no hay análisis de robustez posible. La Tabla 4 resume qué se guarda en cada registro; la interfaz dedica a este histórico una sección con métricas en vivo y exportación a CSV.

Campo del registro	Contenido
id · fecha_hora	Identificador único y marca de tiempo de la ejecución.
consulta · filtros	Qué pidió el usuario y con qué configuración.
modelo · duracion_seg	Proveedor/motor usado y tiempo total de respuesta.
estado · tipo_error	<i>ok/error</i> y error clasificado por tipo (timeout, rate limit, credenciales, máx. pasos, contrato JSON, conexión).
num_pasos ·	Longitud del ciclo y composición (llamadas a herramientas, errores de herramienta, correcciones, cierre).
pasos_por_tipo	
herramientas_invocadas	Secuencia de herramientas usadas (quién fue consultado y en qué orden).
grupos / comparativo / recomendación	Resultado obtenido: equivalentes identificados, opciones comparadas y opción ganadora.
pasos	Traza completa de la decisión (truncada por registro).

Tabla 4. Campos de cada ejecución registrada en la bitácora automática.

Sobre el histórico, la app calcula en vivo las métricas de validación: total de ejecuciones, tasa de éxito, duración promedio, errores agrupados por tipo y herramientas más usadas. Complementariamente, `docs/bitacora_proyecto.md` registra la bitácora del equipo (checklist de catorce hitos mapeados a la rúbrica, medición de línea base contra resultados con la misma métrica, y entradas por sesión con decisiones y evidencia). La interpretación esperada del monitoreo: tasa de éxito estable y alta, errores concentrados en los casos difíciles documentados (CP-03/CP-04) y duración en el orden de segundos; una degradación sostenida señala datos desactualizados o problemas con el proveedor LLM.

Límites conocidos y riesgos

- **Caducidad de datos:** los precios reflejan una captura puntual; el mitigador es la actualización periódica de `data/`, registrada con fecha en metadatos.
- **Autenticidad:** el agente no verifica genuinidad de productos; ante publicaciones genéricas declara incertidumbre. La decisión final de compra es siempre humana.
- **Calificaciones como proxy de calidad:** provienen de plataformas y pueden sesgar contra vendedores nuevos; el comparativo muestra cifras y alternativas, no verdades.
- **Dependencia del proveedor LLM:** si el servicio cae o agota cuota, el sistema informa y se detiene; no degrada silenciosamente hacia respuestas inventadas.

6. Estado actual y trabajo pendiente

Estado: prototipo funcional versionado (commits 228c16a inicial, efab75a de soporte de despliegue y c38dcbb de bitácora), auditado con revisión de cinco ejes (corrección, legibilidad, arquitectura, seguridad, rendimiento) con hallazgos corregidos, con ruta de despliegue documentada y con bitácora automática de ejecuciones para validar el uso real del agente (seis pruebas automatizadas en total, todas en verde).

Pendiente del grupo:

1. Formalizar la contraparte real y medir la línea base cronometrada (misma métrica antes/después).
2. Reemplazar el catálogo de distribuidores por la lista de precios real del negocio cuando esté disponible.

3. Ejecutar los siete casos de prueba con un modelo real, registrando salidas y desviaciones en la tabla de evidencia; completar CP-05 a CP-07.
4. Realizar la sustentación con el entorno preparado (arranque en menos de 60 segundos) usando este documento como guía de estructura para su propia memoria.

Nota metodológica final. Este prototipo declara su uso de IA de forma transparente: el andamiaje del código y de este informe contó con asistencia de IA bajo supervisión directa, las decisiones de diseño fueron revisadas y corregidas humanamente (la auditoría detectó y arregló defectos reales, documentados en el historial del repositorio), y cada integrante debe poder explicar y modificar cualquier parte del sistema. Esa es, precisamente, la honestidad intelectual que la actividad evalúa.

Quedo atento a preguntas y a las reuniones de visto bueno de los grupos.

Javier Mauricio Sierra

Facultad de Estadística
Universidad Santo Tomás
